Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 08-04-2026**	**Enrolment No: 92410133038**

Aim: To understand the constraint satisfaction problem in problem formulation

IDE: Google Colab

Theory:

The Constraint Satisfaction Problem (CSP) is a fundamental concept in computer science and artificial intelligence, playing a crucial role in solving a wide range of real-world problems. At its core, CSP involves finding a solution to a problem where the solution must satisfy a set of constraints that define the problem's requirements and limitations. These constraints restrict the possible values that variables can take, leading to a search for a combination of values that satisfies all constraints simultaneously.

CSPs can be formulated in various domains, including scheduling, planning, resource allocation, configuration, and design. Examples of CSP applications include job scheduling, timetabling, Sudoku puzzles, map coloring, and the traveling salesman problem. In each case, the problem consists of a set of variables, each with a domain of possible values, and a set of constraints that specify allowable combinations of variable values.

The basic components of a CSP include:

1. **Variables**: These represent the entities whose values need to be determined. Each variable has a domain, which is a set of possible values it can take.
2. **Domains**: A domain is the set of values that a variable can take. Domains can be finite or infinite, discrete or continuous, depending on the problem domain.
3. **Constraints**: Constraints define the relationships among variables and restrict the combinations of values they can take. Constraints can be unary (applying to a single variable), binary (between two variables), or higher-order (involving three or more variables).

The goal in solving a CSP is to find an assignment of values to variables such that all constraints are satisfied. This involves searching through the space of possible assignments to find a solution or determine that no solution exists. The search process typically involves backtracking, constraint propagation, and variable ordering heuristics to efficiently explore the search space.

One common approach to solving CSPs is backtracking search, which systematically explores the space of possible assignments by making choices and backtracking when a dead-end is reached. Backtracking search is guided by variable and value ordering heuristics to prioritize the search for promising solutions and avoid exploring unpromising regions of the search space.

Another technique used in CSP solving is constraint propagation, which involves enforcing constraints locally to reduce the domain of variables and prune the search space. Constraint propagation techniques include arc consistency, domain reduction, and constraint propagation algorithms such as AC-3 and AC-4.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 08-04-2026**	**Enrolment No: 92410133038**

In addition to backtracking search and constraint propagation, other methods for solving CSPs include local search algorithms, genetic algorithms, and constraint satisfaction neural networks. Each approach has its strengths and weaknesses, making them suitable for different types of CSPs and problem domains.

Despite the challenges involved in solving CSPs, they offer a powerful framework for modeling and solving complex combinatorial optimization problems. By formulating real-world problems as CSPs and applying appropriate solving techniques, researchers and practitioners can tackle a wide range of practical problems efficiently and effectively.

Methodology:

1. Mention the constraints for sudoku solution

   In the Sudoku problem, a set of constraints must be satisfied to obtain a valid solution. Each row in the grid must contain all numbers from 1 to 9 without any repetition. Similarly, each column must also contain unique numbers from 1 to 9. In addition to rows and columns, each 3×3 sub-grid must include all digits from 1 to 9 exactly once.

   Every empty cell in the Sudoku grid can take values only from the domain {1, 2, 3, ..., 9}. However, the value assigned to a cell must not violate any of the row, column, or sub-grid constraints. These restrictions ensure that the final solution is consistent and valid. Thus, the constraints play a crucial role in limiting the possible values and guiding the solution process.

2. Solve the sudoku using the CSP approach

   The Sudoku problem is solved using the Constraint Satisfaction Problem (CSP) approach with a backtracking algorithm. In this approach, each empty cell is treated as a variable, and the possible values from 1 to 9 form its domain. The algorithm begins by selecting an empty cell and attempts to assign a value from its domain.

   Before assigning a value, the algorithm checks whether the assignment satisfies all Sudoku constraints using a safety-check function. If the value is valid, it is temporarily assigned, and the algorithm proceeds recursively to the next empty cell. If a conflict occurs and no valid value can be assigned, the algorithm performs backtracking by removing the previous assignment and trying a different value.

   This process continues until all cells are filled with values that satisfy all constraints. By systematically exploring possible combinations and eliminating invalid ones, the CSP approach ensures that a correct and complete solution is obtained.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 08-04-2026**	**Enrolment No: 92410133038**

Program (Code):

```
# Function to check if it is safe to place num at mat[row][col]
def isSafe(mat, row, col, num):

    # Check if num exists in the row
    for x in range(9):
        if mat[row][x] == num:
            return False

    # Check if num exists in the col
    for x in range(9):
        if mat[x][col] == num:
            return False

    # Check if num exists in the 3x3 sub-matrix
    startRow = row - (row % 3)
    startCol = col - (col % 3)

    for i in range(3):
        for j in range(3):
            if mat[i + startRow][j + startCol] == num:
                return False

    return True
```

```
# Function to solve the Sudoku problem
def solveSudokuRec(mat, row, col):
    # base case: Reached nth column of the last row
    if row == 8 and col == 9:
        return True

    # If last column of the row go to the next row
    if col == 9:
        row += 1
        col = 0

    # If cell is already occupied then move forward
    if mat[row][col] != 0:
        return solveSudokuRec(mat, row, col + 1)

    for num in range(1, 10):

        # If it is safe to place num at current position
        if isSafe(mat, row, col, num):
            mat[row][col] = num
            if solveSudokuRec(mat, row, col + 1):
                return True
            mat[row][col] = 0
```

```
    return False


def solveSudoku(mat):
    solveSudokuRec(mat, 0, 0)


if __name___== "__main__":
    mat = [
        [3, 0, 6, 5, 0, 8, 4, 0, 0],
        [5, 2, 0, 0, 0, 0, 0, 0, 0],
        [0, 8, 7, 0, 0, 0, 0, 3, 1],
        [0, 0, 3, 0, 1, 0, 0, 8, 0],
        [9, 0, 0, 8, 6, 3, 0, 0, 5],
        [0, 5, 0, 0, 9, 0, 6, 0, 0],
        [1, 3, 0, 0, 0, 0, 2, 5, 0],
        [0, 0, 0, 0, 0, 0, 0, 7, 4],
        [0, 0, 5, 2, 0, 6, 3, 0, 0]
    ]

    solveSudoku(mat)

    for row in mat:
        print(" ".join(map(str, row)))
```

Results:

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 08-04-2026**	**Enrolment No: 92410133038**

To be attached with
Solved sudoku

Result

3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Observation

The given program applies a backtracking-based approach to solve the Sudoku puzzle effectively. During execution, it is observed that the algorithm systematically scans each cell of the grid and identifies empty positions represented by zeros. For every empty cell, the algorithm attempts to assign values from 1 to 9 while ensuring that all Sudoku constraints are satisfied.

The is Safe() function plays a crucial role by verifying whether a number can be placed in a specific position without violating the row, column, and 3×3 sub-grid constraints. If a valid number is found, it is temporarily assigned, and the algorithm proceeds to the next cell recursively.

If at any point no valid number can be assigned to a cell, the algorithm performs backtracking by removing the previously assigned value and trying alternative values. This process continues until a valid configuration is achieved for the entire grid. The observation shows that the algorithm efficiently explores possible solutions while eliminating invalid paths using constraints.

Post Lab Exercise:

Write the code and output for N-Queens problem. Take the value of N from the user.

```
def isSafe(board, row, col, N):
    # Check left side of row
    for i in range(col):
        if board[row][i] == 1:
```

```
        return False

    # Check upper diagonal on left side
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check lower diagonal on left side
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True


def solveNQUtil(board, col, N):
    # Base case: If all queens are placed
    if col >= N:
        return True

    for i in range(N):
        if isSafe(board, i, col, N):
            board[i][col] = 1

            if solveNQUtil(board, col + 1, N):
                return True

            # Backtracking
            board[i][col] = 0

    return False


def solveNQ(N):
    board = [[0 for _ in range(N)] for _ in range(N)]

    if not solveNQUtil(board, 0, N):
        print("No solution exists")
        return
```

```
    print("Solution for", N, "Queens:")
    for row in board:
        print(" ".join("Q" if x == 1 else "." for x in row))


# Taking input from user
N = int(input("Enter value of N: "))
solveNQ(N)
```

Output:-

```
Enter value of N: 4

Solution for 4 Queens:
. . Q .
Q . . .
. . . Q
. Q . .
```